

Secondary Index

1 What is a Secondary Index?

A **Secondary Index** is used when the data file is physically **Unordered / Unsorted** on disk. It provides a secondary search path for query retrieval without altering the physical order of data records.

★ GOLDEN RULE

Secondary Index = Unordered Data File

2 Complete DBMS Indexing Matrix 🔥

Data File Ordering	Search Field Type	Index Type Classification	Density Type
Ordered (Sorted)	Key (Unique)	🟢 Primary Index	Sparse
Ordered (Sorted)	Non-Key (Duplicates)	🟡 Clustered Index	Sparse
Unordered (Unsorted)	Key (Unique)	🟠 Secondary Index (Key)	Dense
Unordered (Unsorted)	Non-Key (Duplicates)	🟠 Secondary Index (Non-Key)	Dense

3 Why Do We Need a Secondary Index?

Suppose an employee table is physically ordered by Employee ID (using a Primary Index):

Employee ID (Primary Key)	Name	PAN Number
1	Ram	ABC23
2	Ali	XYZ45
3	John	PQR67

Querying WHERE Employee ID = 2 uses the Primary Index efficiently. But for WHERE PAN = 'XYZ45' or WHERE Name = 'Ali', the primary ordering is useless. A **Secondary Index** on PAN or Name creates an alternate sorted search path.

4 Case 1: Secondary Index on a Key (Unique) 🔑

When the search field is **Unique** but the underlying data file is **Unordered**:

Unordered Data Records on Disk

Record 1: PAN = 23
Record 2: PAN = 62
Record 3: PAN = 33
Record 4: PAN = 40
Record 5: PAN = 51

Sorted Secondary Index Table

23 → Pointer to Record 1
33 → Pointer to Record 3
40 → Pointer to Record 4
51 → Pointer to Record 5
62 → Pointer to Record 2

Secondary Index on Key is DENSE: Number of Index Entries = Number of Data Records

5 Why Can't We Use Sparse Index for Unordered Data?

6 Case 2: Secondary Index on Non-Key (Duplicates Allowed)

When the search field is **Unordered** and contains **Duplicate Values** (e.g., Name: A, B, A, C, A):

Creating duplicate index entries ($A \rightarrow \text{Ptr1}$, $A \rightarrow \text{Ptr2}$, $A \rightarrow \text{Ptr3}$) bloats the index. Instead, an **Intermediate Layer (Block of Record Pointers)** is introduced:

Two-Level Architecture for Non-Key Secondary Index:

```
[ Secondary Index ] —> [ Block of Record Pointers ] —> [ Data Records ]
| Key: A | Pointer | —> | Ptr to Rec 1 (A) | —> Record 1 (A)
| Key: B | Pointer | | Ptr to Rec 3 (A) | —> Record 3 (A)
|                                     | Ptr to Rec 7 (A) | —> Record 7 (A)
```

Because every individual occurrence of duplicate values ultimately gets a record pointer in the intermediate level, a Non-Key Secondary Index is also considered **DENSE** overall.

7 Time Complexity Analysis

Secondary Index Type	Search Execution Steps	Total Block Accesses
Secondary Index (Key)	Binary Search on Index + Access Target Data Block	$\log_2 N + 1$
Secondary Index (Non-Key)	Binary Search on Index + Intermediate Pointer Block + Access Data Blocks	$\log_2 N + \text{Extra Pointer/Data Accesses}$

8 Master Comparison: Primary vs. Clustered vs. Secondary

Feature	Primary Index	Clustered Index	Secondary Index
Data File Ordered?	Yes	Yes	No
Search Field	Key (Unique)	Non-Key (Duplicates)	Key OR Non-Key
Index Density	Sparse	Sparse	Dense
Index Sorted?	Yes	Yes	Yes (Index itself is sorted)
Anchor Record?	Yes	Yes	No
Extra Layer Needed?	No	May need Block Hanger	Intermediate Pointer Block (Non-Key)

9 Most Important GATE / UGC-NET Exam Points 🔥

- **Rule 1:** Secondary Index is ALWAYS required when the data file is unordered.
- **Rule 2:** Secondary Index is ALWAYS **Dense** (whether key or non-key).
- **Rule 3:** The secondary index entries are kept strictly **sorted** to enable Binary Search, even though underlying data is unordered.
- **Rule 4:** Non-Key secondary indexing uses an **intermediate block of record pointers** to handle duplicates efficiently.

🧠 10-Second Revision Memory Chain

Primary → Ordered + Key → Sparse | Clustered → Ordered + Non-Key → Sparse | Secondary → Unordered → Dense